ShopNow Support Assistant — Project Guideline

An AI customer-support assistant that answers only from the company's own documents

1 What is this project?

ShopNow Support Assistant answers customer questions about orders, returns, shipping, payments, warranties, products and consumer rights. It uses a method called **RAG (Retrieval-Augmented Generation)**: before the AI answers, the system **looks up the relevant passages** in ShopNow's own documents and the AI must answer **only from those passages**, citing them. This keeps answers accurate and in line with the current documents, and avoids made-up facts. The customer's region (US, EU or UK) decides which legal rules apply.

14	61	3	3
source documents*	searchable passages	answer pipelines	regions: US · EU · UK

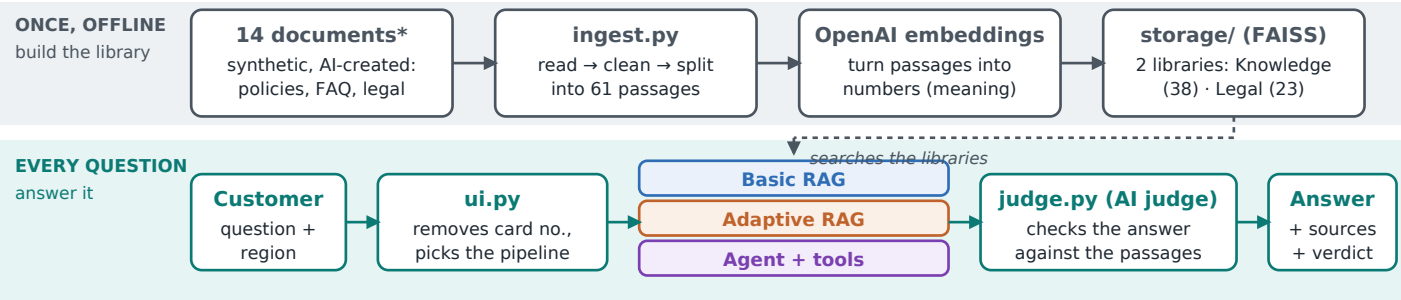
* **Disclaimer:** all 14 source documents (the ShopNow policies, FAQ, product guides, terms, privacy policy and consumer-rights pages) are **synthetic data created by AI** for this demo. ShopNow is a fictional store; the documents are not real company policy or legal advice.

2 Technology used

Python	The language the whole project is written in.	**FAISS**	Fast search library that finds the passages closest in meaning to a question.
LangChain	Connects the pieces: prompts, AI models, search and tools.	**Gradio**	The simple web page where you type a question and see the answer.
OpenAI GPT-4.1	Writes the answers and acts as the judge.	**Pydantic**	Strict format for the judge's verdict and the AI-written search queries.
OpenAI GPT-4.1-mini	Cheaper model that rewrites or splits questions into better searches.	**Debug logger**	Writes every step, AI prompt and AI reply of a session to a log file.
OpenAI embeddings	Turns text into numbers so we can search by meaning, not exact words.		

3 How it works

The documents are prepared **once** into two searchable libraries. Then, for every question, the chosen pipeline finds the right passages, the AI writes an answer, and an AI judge checks it (on by default) before the customer sees it.



4 What the project shows (outcome)

■ **Grounded answers** with sources, using only approved company documents. ■ **Quality control**: an AI judge blocks wrong or unsupported answers. ■ **Three designs compared** on the same data, so we can see when the simple approach is enough and when a smarter one is worth the extra cost. ■ **Full traceability**: every step, AI prompt and AI reply is recorded in a log file for review.

5 Built-in safeguards

■ **AI judge** checks the answer: is the right information there, does it answer the question, is every fact backed by the documents? Score 4/5 or better to pass. Always on in Adaptive RAG; on by default (a tick box) in Basic RAG and the Agent.

■ **Safe fallback:** if no answer passes the judge, the customer is pointed to support instead of a guess.

■ **Privacy:** card numbers are removed before anything is sent to OpenAI.

■ **Region aware:** searches only return passages marked for all regions or for the customer's own region, so EU-only and UK-only legal pages are never used for a US customer.

6 The three answer pipelines

1 · Basic RAG

Simple, fixed recipe

Searches both libraries with the customer's own words, gives the best 4 passages from each library (8 in total) to the AI, and the AI writes one answer.

Best for: Clear, single-topic questions
Cost: Fastest and cheapest

2 · Advanced (Adaptive) RAG

Changes its search when needed

First checks the question: several parts? weak match? Then starts with one search plan: as-is, reworded, or split into sub-questions. If the judge rejects the answer, it tries another plan, including split + reworded together (up to 3 tries).

Best for: Vague wording or several questions in one
Cost: Cheap when easy, pays more only when needed

3 · Agent with tool calling

The AI decides what to do

The AI gets tools: search the knowledge library, search the legal library, calculate a date, request a human (a placeholder in this demo: it only replies that support will email). It chooses which to use, with what words, and repeats until it can answer (up to 5 rounds).

Best for: Multi-step questions and deadlines
Cost: Most flexible, more AI calls

7 Three journeys for one question: function call chains

Example: a US customer asks **"How much does standard shipping cost?"** The answer is in one passage of the Shipping policy: *standard shipping is $5.99, free on orders of $50 or more; express is $14.99 in selected areas.* Each block shows the expected chain of function calls, with the file on the right. ◆ = call to an OpenAI chat model (GPT-4.1) · ★ = OpenAI embedding call (turns search words into numbers).

Common entrance (all three pipelines)

```
[Ask] → answer_question(question, region, mode, use_judge)                                   ui.py
        └─ _answer_question()
           ├─ remove_card_numbers(question)                                                  preprocess.py
           ├─ get_llm()  → GPT-4.1 client (no call yet)                                      models.py
           ├─ get_vector_stores() → load_vector_store("knowledge"), ("legal")  (first time only)   vector_store.py
           ├─ create_qa_prompt() → create_qa_chain(llm, prompt)                              prompts.py · rag_chain.py
           └─ branch on mode → A · B · C
```

A · Basic RAG

```
generate_answer(question, stores, qa_chain, "US")                                            rag_chain.py
├─ retrieve(question, stores["knowledge"], "US")  ★ → best 4 chunks                          retriever.py
├─ retrieve(question, stores["legal"], "US")      ★ → best 4 chunks
├─ format_context(8 chunks) → numbered text                                                  retriever.py
└─ qa_chain.invoke({region, context, question, feedback: ""})  ◆ → answer                    rag_chain.py
create_judge(get_judge_llm())  (only if "Use LLM judge" is ticked)                           judge.py · models.py
└─ verify_answer(question, context, answer, judge, qa_chain, "US")                           judge.py
   ├─ judge_answer()  ◆ → is_approved() → True → send the answer
   └─ if rejected: context not relevant → safe message · else rewrite once ◆ + judge again ◆
return → format_sources() · format_verdict()                                                 ui.py · judge.py
Hand-off to a human: not available (no tools).                                               2 ◆ · 2 ★
```

B · Advanced (Adaptive) RAG

```
get_fast_llm() · create_judge(get_judge_llm()) · create_expander() · create_decomposer()     models.py · judge.py ·
                                                                                             query_strategies.py
answer_adaptively(question, stores, qa_chain, judge, expander, decomposer, "US")             adaptive.py
├─ choose_first_strategy(question, stores, "US")                                             strategy_selector.py
│  ├─ is_multi_part(question) → False (one "?", no joining words, 6 words)
│  └─ best_distance(question, stores, "US")  ★★ → strong match → "direct"
├─ attempt 1 of up to 3
│  ├─ build_queries(question, "direct") → [question]   (no AI call)                          query_strategies.py
│  ├─ retrieve_for_queries() → retrieve() ★ per library → remove duplicates → 8 chunks       query_strategies.py ·
│                                                                                            retriever.py
│  ├─ format_context(8 chunks)                                                              retriever.py
│  ├─ qa_chain.invoke({…, feedback: ""})  ◆ → answer                                        rag_chain.py
│  └─ judge_answer()  ◆ → is_approved() → True → return       (judge always on here)        judge.py
└─ if rejected: next_strategy() → expansion / decomposition / both → search again (max 3)    strategy_selector.py
return → format_sources() · format_attempts() · format_verdict()                             ui.py · adaptive.py
Hand-off to a human: not available (no tools).                                               2 ◆ · 4 ★
```

C · Agent with tool calling

```
create_tools(stores, "US", documents)                                                        agent.py
   → search_knowledge_base · search_legal_base · calculate_date · escalate_to_human
run_agent(question, llm, tools, create_agent_prompt(), "US")                                 agent.py · prompts.py
├─ llm.bind_tools(tools) · prompt.format_messages()
├─ round 1: llm_with_tools.invoke(messages)  ◆ → tool call search_knowledge_base("standard shipping cost")
│     └─ retrieve(query, stores["knowledge"], "US") ★ → 4 chunks → format_context() → to AI   retriever.py
└─ round 2: llm_with_tools.invoke(messages)  ◆ → final answer, no tool call → return  (max 5 rounds)
create_judge(get_judge_llm()) → verify_answer(...) → judge_answer()  ◆ → approved            judge.py
return → format_sources() · tool steps · format_verdict()                                    ui.py
escalate_to_human(summary): NOT used for this question. The AI calls it only when the       agent.py
   customer asks for a person or has a dispute → fixed reply "a human support agent will
   email you" (placeholder: no ticket or email yet). No search → judge skipped.              3 ◆ · 1 ★
```